

C++ STL Cheat Sheet — Quick Lookup

*Focus: containers, algorithms, and the part everyone forgets — **custom comparators**. Headers shortcut: `#include <bits/stdc++.h>` using `namespace std;`*

1. Containers — Quick Reference

Container	Header	Ordered?	Duplicates?	Access	Insert/Erase	Use When
<code>vector<T></code>	<code><vector></code>	Insertion order	Yes	$O(1)$ random	$O(1)$ end, $O(n)$ middle	Default dynamic array
<code>array<T,N></code>	<code><array></code>	Insertion order	Yes	$O(1)$	Fixed	Fixed-size known at compile time
<code>deque<T></code>	<code><deque></code>	Insertion order	Yes	$O(1)$ random	$O(1)$ both ends	Need <code>push_front</code> + <code>push_back</code>
<code>list<T></code>	<code><list></code>	Insertion order	Yes	$O(n)$	$O(1)$ anywhere (with iter)	Frequent middle insert/erase
<code>stack<T></code>	<code><stack></code>	LIFO	Yes	<code>top()</code>	<code>push/pop</code>	LIFO
<code>queue<T></code>	<code><queue></code>	FIFO	Yes	<code>front()/back()</code>	<code>push/pop</code>	FIFO / BFS
<code>priority_queue<T></code>	<code><queue></code>	Heap	Yes	<code>top()</code> $O(1)$	<code>push/pop</code> $O(\log n)$	Max-heap by default
<code>set<T></code>	<code><set></code>	Sorted	No	$O(\log n)$	$O(\log n)$	Sorted unique elements
<code>multiset<T></code>	<code><set></code>	Sorted	Yes	$O(\log n)$	$O(\log n)$	Sorted, allow dups
<code>map<K,V></code>	<code><map></code>	Sorted by key	Unique keys	$O(\log n)$	$O(\log n)$	Sorted key→value
<code>multimap<K,V></code>	<code><map></code>	Sorted by key	Yes	$O(\log n)$	$O(\log n)$	Multiple values per key
<code>unordered_set<T></code>	<code><unordered_set></code>	No	No	$O(1)$ avg	$O(1)$ avg	Hash, just need exists check
<code>unordered_map<K,V></code>	<code><unordered_map></code>	No	Unique keys	$O(1)$ avg	$O(1)$ avg	Fastest key→value

2. Vector — The Workhorse

```

vector<int> v; // empty
vector<int> v(5, 0); // size 5, all 0
vector<int> v = {1, 2, 3};
vector<vector<int>> grid(n, vector<int>(m, 0)); // n×m matrix

v.push_back(x); v.pop_back();
v.emplace_back(x); // constructs in-place, avoids a copy – prefer for objects/pairs
v.size(); v.empty(); v.clear();
v.front(); v.back();
v[i]; // no bounds check (UB if out of range)
v.at(i); // bounds-checked, throws std::out_of_range
v.insert(v.begin()+i, x);
v.erase(v.begin()+i);
v.erase(v.begin()+l, v.begin()+r); // range erase [l, r)
v.resize(n); v.resize(n, val); // grow/shrink, new elements default/val-init
v.reserve(n); // pre-allocate capacity, avoids reallocation
v.capacity(); // current allocated capacity (>= size())
v.shrink_to_fit(); // request capacity == size()
swap(v1, v2); // O(1) – swaps internal buffers

// iterate
for (int x : v) cout << x;
for (int i = 0; i < v.size(); i++) ...;
for (auto it = v.rbegin(); it != v.rend(); it++) ...; // reverse iteration

// 2D vector traversal
for (auto& row : grid)
    for (int x : row) ...;

// useful
reverse(v.begin(), v.end());
sort(v.begin(), v.end());
auto it = find(v.begin(), v.end(), x);
int cnt = count(v.begin(), v.end(), x);
int sum = accumulate(v.begin(), v.end(), 0);
int mx = *max_element(v.begin(), v.end());
int mn = *min_element(v.begin(), v.end());

```

3. String — Often-forgotten Bits

```
string s = "hello";
s.length(); s.size();
s.substr(pos, len);
s.find("ll"); // returns index or string::npos
s += "world";
s.push_back('x'); s.pop_back();
reverse(s.begin(), s.end());
sort(s.begin(), s.end());

// number ↔ string
int n = stoi("123");
long long ll = stoll("123456789012");
string t = to_string(42);

// char checks
isdigit(c); isalpha(c); isalnum(c); isupper(c); islower(c);
tolower(c); toupper(c);

// compare
s.compare(t); // 0 if equal, <0 if s<t, >0 if s>t
s == t; s < t; // lexicographic, usually clearer than compare()

// split on whitespace with stringstream
#include <sstream>
stringstream ss("the quick fox");
string word;
vector<string> words;
while (ss >> word) words.push_back(word);

// split on a delimiter
stringstream ss2("a,b,c");
string token;
while (getline(ss2, token, ',')) words.push_back(token);
```

4. Set / Map

```
set<int> s;
s.insert(x); s.erase(x); s.count(x); // count returns 0/1
s.emplace(x); // constructs in-place, avoids a copy
auto it = s.find(x); // s.end() if not found

// Lower_bound / upper_bound (work in O(log n) for set/map)
auto it = s.lower_bound(x); // first >= x
auto it = s.upper_bound(x); // first > x

// min / max element without extra search – set/map are sorted
int mn = *s.begin();
int mx = *s.rbegin(); // reverse iterator, 0(1)

map<string, int> mp;
mp["apple"] = 3;
mp["apple"]++; // creates with 0 if absent then ++
mp.emplace("banana", 2); // does NOT overwrite if key exists
if (mp.count("apple")) ...;
for (auto& [k, v] : mp) cout << k << v; // C++17 structured binding
mp.erase("apple");
```

⚠ `unordered_map` / `unordered_set` do **NOT** have `lower_bound` / `upper_bound` .

⚠ `multiset::erase(value)` removes **ALL** copies of `value` in $O(\log n + \text{count})$. To erase just one occurrence:

```
s.erase(s.find(value));
```

5. Stack / Queue / Deque

```
stack<int> st;      st.push(x); st.pop(); st.top();
queue<int> q;       q.push(x); q.pop(); q.front(); q.back();
deque<int> dq;      dq.push_front(x); dq.push_back(x);
                  dq.pop_front(); dq.pop_back();
                  dq.front(); dq.back();
```

6. Priority Queue — MAX HEAP vs MIN HEAP

Default — MAX HEAP (largest on top)

```
priority_queue<int> pq;           // MAX heap
pq.push(3); pq.push(1); pq.push(5);
pq.top();                         // 5
pq.pop();
```

MIN HEAP (smallest on top)

```
priority_queue<int, vector<int>, greater<int>> pq; // MIN heap
pq.push(3); pq.push(1); pq.push(5);
pq.top();                                         // 1
```

Rule of thumb: - **Max heap** → "I want the largest so far / biggest k elements / scheduling longest task first." - **Min heap** → "I want the smallest so far / Dijkstra / merge K sorted / k th largest (keep top k smallest popped)."

Trick: **k th largest** = min heap of size k . **k th smallest** = max heap of size k .

7. CUSTOM COMPARATORS — The Confusing Part

There are **3 ways** to write a comparator. The rules differ by container.

The Mental Model

For `sort` / `set` / `map` : comparator returns `true` if `a` should come **before** `b` . For `priority_queue` : comparator returns `true` if `a` has **lower priority** than `b` (i.e., `a` goes deeper). It is the **opposite** of `sort` — this is why people get confused.

sort ascending:	return a < b;	→ smaller first
priority_queue:	return a < b;	→ MAX heap (larger has priority)
priority_queue:	return a > b;	→ MIN heap (smaller has priority)

7a. `sort` with custom comparator

Lambda (cleanest, most common in interviews):

```
vector<pair<int,int>> v;
// sort by second ascending
sort(v.begin(), v.end(), [](auto& a, auto& b){
    return a.second < b.second;
});

// sort by first descending, ties by second ascending
sort(v.begin(), v.end(), [](auto& a, auto& b){
    if (a.first != b.first) return a.first > b.first;
    return a.second < b.second;
});
```

Free function:

```
bool cmp(const pair<int,int>& a, const pair<int,int>& b) {
    return a.second < b.second;
}
sort(v.begin(), v.end(), cmp);
```

Functor (struct with `operator()`):

```
struct Cmp {
    bool operator()(const pair<int,int>& a, const pair<int,int>& b) const {
        return a.second < b.second;
    }
};
sort(v.begin(), v.end(), Cmp());
```

Concrete **sort** examples

```
// 1) Sort strings by Length
sort(words.begin(), words.end(), [](const string& a, const string& b){
    return a.size() < b.size();
});

// 2) Sort intervals by start time
vector<vector<int>>> intervals;
sort(intervals.begin(), intervals.end(), [](auto& a, auto& b){
    return a[0] < b[0];
});

// 3) Sort by absolute value
sort(v.begin(), v.end(), [](int a, int b){
    return abs(a) < abs(b);
});

// 4) Descending sort (built-in)
sort(v.begin(), v.end(), greater<int>());
```

7b. **priority_queue** with custom comparator

For PQ you usually need a **functor (struct)** because the type is part of the template.

```
struct Cmp {
    bool operator()(const pair<int,int>& a, const pair<int,int>& b) const {
        return a.second > b.second;    // smaller .second has priority → MIN by .second
    }
};
priority_queue<pair<int,int>, vector<pair<int,int>>, Cmp> pq;
```

Lambda version (C++11+):

```
auto cmp = [](const pair<int,int>& a, const pair<int,int>& b){
    return a.second > b.second;    // MIN heap by .second
};
priority_queue<pair<int,int>, vector<pair<int,int>>, decltype(cmp)> pq(cmp);
```

Concrete PQ examples

```
// 1) Min heap of ints (built-in greater)
priority_queue<int, vector<int>, greater<int>> minHeap;

// 2) Max heap of pairs by .first (default works because pair<,> compares lexicographically)
priority_queue<pair<int,int>> maxHeap;
// Top = pair with Largest first, ties → Largest second.

// 3) Min heap of pairs by .first → use greater<>
priority_queue<pair<int,int>, vector<pair<int,int>>, greater<>> minHeap;

// 4) Custom: tasks {priority, id}; want HIGHEST priority first, ties → SMALLEST id first
struct TaskCmp {
    bool operator()(const pair<int,int>& a, const pair<int,int>& b) const {
        if (a.first != b.first) return a.first < b.first; // Larger priority first
        return a.second > b.second; // smaller id first on ties
    }
};
priority_queue<pair<int,int>, vector<pair<int,int>>, TaskCmp> tasks;

// 5) Kth Largest element in array – use MIN heap of size k
priority_queue<int, vector<int>, greater<int>> mh;
for (int x : nums) {
    mh.push(x);
    if (mh.size() > k) mh.pop();
}
// mh.top() is the kth Largest
```

7c. `set` / `map` with custom comparator

```
// set sorted in DESCENDING order
set<int, greater<int>> s;

// set of pairs sorted by .second
struct Cmp {
    bool operator()(const pair<int,int>& a, const pair<int,int>& b) const {
        if (a.second != b.second) return a.second < b.second;
        return a.first < b.first; // tie-break to keep set well-defined
    }
};
set<pair<int,int>, Cmp> s;
```

△ For `set` / `map`, the comparator must define a **strict weak ordering**. `cmp(a,a)` must be false. Always include a tie-breaker if your initial key has duplicates you want to keep distinct.

7d. `unordered_map` with custom KEY (e.g., pair)

`unordered_map<pair<int,int>, int>` does **not** compile out of the box — no hash for pair.

Quick fix for interviews:


```

struct PairHash {
    size_t operator()(const pair<int,int>& p) const {
        return hash<long long>()(((long long)p.first << 32) ^ (unsigned)p.second);
    }
};
unordered_map<pair<int,int>, int, PairHash> mp;

```

Or just use `map<pair<int,int>, int>` (works directly, $O(\log n)$).

8. Algorithms — The Important Ones

```

sort(v.begin(), v.end());
sort(v.begin(), v.end(), greater<int>()); // descending
stable_sort(v.begin(), v.end()); // preserves equal-element order
reverse(v.begin(), v.end());

// Binary search (sorted range only)
bool found = binary_search(v.begin(), v.end(), x);
auto lb = lower_bound(v.begin(), v.end(), x); // first >= x
auto ub = upper_bound(v.begin(), v.end(), x); // first > x
int idx = lb - v.begin();
int countOfX = upper_bound(v.begin(), v.end(), x)
    - lower_bound(v.begin(), v.end(), x);

// numeric
accumulate(v.begin(), v.end(), 0LL);
*max_element(v.begin(), v.end());
*min_element(v.begin(), v.end());
fill(v.begin(), v.end(), 0);
iota(v.begin(), v.end(), 1); // fills with 1,2,3,...

// permutations
next_permutation(v.begin(), v.end());
prev_permutation(v.begin(), v.end());

// unique (must be sorted first)
sort(v.begin(), v.end());
v.erase(unique(v.begin(), v.end()), v.end());

// gcd / lcm (C++17)
__gcd(a, b); gcd(a, b); lcm(a, b);

// predicate-based search
auto it = find_if(v.begin(), v.end(), [](int x){ return x % 2 == 0; });
int cnt = count_if(v.begin(), v.end(), [](int x){ return x > 0; });
bool all = all_of(v.begin(), v.end(), [](int x){ return x > 0; });
bool any = any_of(v.begin(), v.end(), [](int x){ return x < 0; });
bool none = none_of(v.begin(), v.end(), [](int x){ return x < 0; });

// transform - apply a function elementwise into another (or the same) container
vector<int> sq(v.size());
transform(v.begin(), v.end(), sq.begin(), [](int x){ return x * x; });

// bit tricks (GCC builtins - very common in CP, undefined for x == 0 except popcount)
__builtin_popcount(x); // number of set bits (int)
__builtin_popcountll(x); // Long Long version
__builtin_ctz(x); // count trailing zeros
__builtin_clz(x); // count leading zeros

```

9. Pair / Tuple

```
pair<int,string> p = {1, "abc"};
p.first; p.second;
auto [a, b] = p; // C++17

tuple<int,int,string> t = {1, 2, "x"};
get<0>(t); get<2>(t);
auto [x, y, z] = t;
```

10. Common Patterns You Should Recognize

```
// Frequency map
unordered_map<int,int> freq;
for (int x : v) freq[x]++;

// Two-sum style
unordered_map<int,int> idx;
for (int i = 0; i < n; i++) {
    if (idx.count(target - v[i])) return {idx[target-v[i]], i};
    idx[v[i]] = i;
}

// Sliding window max - deque
deque<int> dq; // stores indices, decreasing values

// BFS
queue<int> q; q.push(start);
vector<bool> seen(n); seen[start] = true;
while (!q.empty()) { int u = q.front(); q.pop(); /* ... */ }

// Dijkstra (min-heap of {dist, node})
priority_queue<pair<int,int>, vector<pair<int,int>>, greater<>> pq;

// Top-K largest → min heap of size k
// Top-K smallest → max heap of size k
// Median of stream → max heap (lower half) + min heap (upper half)
```

11. Quick "Which Heap?" Decision Table

Problem	Heap
Kth largest element / Top-K largest	Min heap of size k
Kth smallest element / Top-K smallest	Max heap of size k
Dijkstra / shortest path	Min heap
Schedule task with highest priority first	Max heap
Merge K sorted lists	Min heap
Median of running stream	Both (max for lower half, min for upper half)
Connect ropes / Huffman code	Min heap
Maximum CPU load / running max	Max heap

12. `bitset<N>` — Fixed-Size Bit Array

```
#include <bitset>
bitset<32> b;           // all zeros, size fixed at compile time
bitset<32> b2(13);      // from integer: 00...01101
bitset<8> b3("1010");  // from string, right-aligned: 00001010

b[i] = 1;   b.set(i);   // set bit i
b.reset(i); // clear bit i
b.flip(i);  // toggle bit i
b.flip();   // toggle all bits

b.count(); // number of set bits
b.any(); b.none(); b.all(); // any set? none set? all set?
b.to_string(); // as a string
b.to_ulong(); b.to_ullong(); // as an unsigned integer

// supports bitwise ops
bitset<8> a("1100"), c("1010");
a & c; a | c; a ^ c; ~a; a << 1; a >> 1;
```

13. Last-Minute Gotchas

- `priority_queue` comparator logic is **inverted** vs `sort` — if confused, write it out as "the one that returns true goes deeper / has lower priority."
- `unordered_map` is faster average but worst-case $O(n)$. For adversarial inputs use `map`.
- `set::erase(it)` is $O(1)$ amortized if you have the iterator; `set::erase(value)` is $O(\log n)$.
- `vector::erase` invalidates iterators after the erased element.
- Use `reserve(n)` on a vector if you know the size — avoids reallocation.

- `auto it = mp.find(key); if (it != mp.end()) ...` — avoid double-lookup vs `mp[key]` then `mp.count`.
 - `mp[key]` **inserts** a default value if key absent — use `count` or `find` for read-only checks.
 - Integer overflow: `1LL * a * b` not `a * b` when product can exceed `int`.
-